جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

Data Engineering & AI Engineering

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره



جلسه ششم:SSH, Git, Docker و PostgreSQL

مدرس :مصطفی قدیمی

تاریخ :یکشنبه ۲۶ مهر ۱۴۰۵

تاریخ به روز رسانی : ۲۰۲۶۱۰۲۷

درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

مخزن جزوه‌ها:

https://github.com/LaneZero/data-ai-engineering-course

وب‌سایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

فهرست مطالب

هدف جلسه و مسئله اصلی .Error! Bookmark not defined

مسیر محصول و درس تجربه موری .Error! Bookmark not defined

فرایند تصمیم‌گیری داده‌محور .Error! Bookmark not defined

هزینه بازگشت و نقش فرضیه‌ها .Error! Bookmark not defined

انتخاب متریک درست برای رشد .Error! Bookmark not defined

Funnel، Conversion و متریک‌های سلامت .Error! Bookmark not defined

Retentionو نگاه.Cohort Error! Bookmark not defined

اقتصاد مشتری Payback: و.LTV Error! Bookmark not defined

وقتی داده گمراه می‌کند .Error! Bookmark not defined

تیم داده را بر اساس تصمیم واقعی بسازیم .Error! Bookmark not defined

ایجنت‌ها در عملیات شرکت .Error! Bookmark not defined

تمرین عملی و جمع‌بندی .Error! Bookmark not defined

تکمیل‌های فنی .Error! Bookmark not defined

واژه‌نامه .Error! Bookmark not defined

سوال های مرور: .Error! Bookmark not defined

پاسخ‌نامه مختصر .Error! Bookmark not defined

۱. معرفی جلسه و اهداف یادگیری

این جلسه برای پیوند دادن پیش‌نیازهای چند جلسه گذشته برگزار شد. مدرس به‌جای معرفی فهرستی از ابزارهای تازه، نشان داد Linux، شبکه، SSH, Git, GitHub، ماشین مجازی، Docker و PostgreSQL در یک فرایند واقعی چگونه به هم مربوط می‌شوند. بخش مهم کلاس اجرای زنده و رفع خطا بود؛ بنابراین در این جزوه، نتیجه آزمایش‌ها و علت خطاها از توضیح نظری جدا نشده‌اند.

در پایان جلسه انتظار می‌رود بدانیم چرا برای اداره سرور از Terminal استفاده می‌کنیم، فرق ورود با رمز عبور و کلید SSH چیست، چطور تغییرهای Git از سیستم شخصی به GitHub می‌رسند، هنگام ناسازگاری تغییرات چه کاری باید کرد و چرا بالا آمدن کانتینر پایگاه داده به معنای برقرار بودن اتصال شبکه‌ای به آن نیست.

۲. چرا Linux و Command Line پیش‌نیاز کار داده‌اند؟

۲.۱ از رایانه شخصی تا سرور

مدرس بحث را از تفاوت استفاده روزمره از رایانه و استفاده سازمانی از سرور آغاز کرد. لپ‌تاپ شخصی ممکن است روشن یا خاموش شود و کاربر معمولاً با پنجره‌ها، ماوس و محیط گرافیکی کار کند. در مقابل، سرویس‌های اینترنتی و داده‌ای روی سرورهایی اجرا می‌شوند که معمولاً برای ارائه خدمت مداوم طراحی شده‌اند. این سرورها در مراکز داده نگهداری می‌شوند و ممکن است به‌صورت مجموعه‌ای هماهنگ یا Cluster فعالیت کنند.

وقتی بخواهیم روی چندین سرور نرم‌افزار نصب کنیم، تنظیمات را تغییر دهیم یا سرویس‌ها را بررسی کنیم، اتصال گرافیکی به تک‌تک آن‌ها همیشه انتخاب مناسبی نیست. Command Line دسترسی مستقیم‌تر و امکان خودکارسازی فعالیت‌های تکراری را فراهم می‌کند. مدرس تأکید کرد که محیط گرافیکی روی سرور ناممکن نیست؛ برای نمونه روی برخی سرورهای Windows از Remote Desktop Protocol یا RDP استفاده می‌شود.

مفهوم	کاربرد در این جلسه	مثال
Server	اجرای سرویس برای کاربران یا سامانه‌های دیگر	ماشین مجازی اجاره‌شده
Cluster	گروهی از ماشین‌ها که برای هدف مشترکی همکاری دارند	زیرساخت یک خدمت اینترنتی
CLI / Terminal	اجرای دستور و اداره سیستم	`ls` و `cat`

مفهوم	کاربرد در این جلسه	مثال
RDP	دسترسی گرافیکی از راه دور	اتصال به دسکتاپ سرور Windows
SSH	دسترسی امن به خط فرمان از راه دور	اداره سرور Linux

- **تکمیل فنی:** «همیشه روشن بودن» هدف معمول سرویس‌دهی است، نه تضمین اینکه یک سرور هیچ‌وقت خاموش یا بازراه‌اندازی نمی‌شود. نگهداری، خرابی، جابه‌جایی بار و طراحی High Availability می‌توانند موجب توقف یا انتقال سرویس شوند.

۲.۲ ارتباط SSH با مبانی شبکه

برای اتصال به یک ماشین راه دور، باید مقصد را از نظر شبکه بشناسیم. مدرس سروری مجازی ساخت و نشان داد که داشتن IP، نام کاربری و روش احراز هویت، مقدمات اتصال‌اند. اجرای دستور SSH از روی رایانه شخصی باعث می‌شود پس از برقراری اتصال، دستورات در محیط سرور اجرا شوند، نه روی دستگاه محلی.

فرم عمومی دستور در مثال کلاس، تعیین نام کاربری و مقصد بود:

```
ssh USER@SERVER_HOST
```

در اولین اتصال ممکن است SSH اثرانگشت کلید میزبان (Host Key Fingerprint) را نمایش دهد. مدرس این مرحله را در اجرای زنده نشان داد و بعد وارد پوسته سرور شد. پس از ورود، اجرای `ls`، بررسی مسیرها و ساختن فایل، همان تمرین‌های ترمینال جلسه‌های قبل بودند، با این تفاوت که این بار روی ماشین راه دور انجام می‌شدند.

- **تکمیل فنی:** قبل از پذیرش Host Key ناشناس، اثرانگشت را با اطلاعات معتبر ارائه‌دهنده سرور یا مدیر زیرساخت تطبیق دهید. تأیید کورکورانه این پیام، کنترل هویت میزبان را بی‌اثر می‌کند. SSH معمولاً از پورت TCP شماره `22` استفاده می‌کند، اما ممکن است پورت دیگری تنظیم شده باشد.

۳. احراز هویت با کلید SSH

۳.۱ تفاوت Public Key و Private Key

در ابتدای نمایش، مدرس با نام کاربری و رمز عبور به سرور متصل شد. سپس توضیح داد که می‌توان به‌جای وارد کردن رمز در هر نوبت، از زوج کلید استفاده کرد. این زوج از یک کلید عمومی (Public Key) و یک کلید خصوصی (Private Key) تشکیل می‌شود. کلید عمومی روی سرور قرار می‌گیرد و کلید خصوصی روی دستگاه صاحب آن باقی می‌ماند.

فایل یا مفهوم	محل معمول	ملاحظه مهم
Private Key	دستگاه صاحب کلید	هرگز در چت یا مخزن عمومی منتشر نشود
Public Key	دستگاه صاحب کلید و سرور مقصد	اشتراک‌گذاری آن برای اعطای دسترسی مجاز است
`authorized_keys`	پوشه `.ssh` حساب کاربری روی سرور	کلیدهای عمومی مجاز برای ورود را نگه می‌دارد
Passphrase	محافظت از کلید خصوصی	استفاده از آن به امنیت کلید کمک می‌کند

مدرس فایل `authorized_keys` را باز کرد و کلید عمومی خودش و برخی شرکت‌کنندگان را به آن افزود تا اعضای کلاس بتوانند به ماشین آزمایشی وصل شوند. هدف این تمرین، مشاهده عملی ورود به یک سرور مشترک بود.

- **تکمیل فنی:** دادن دسترسی `root` مشترک به چند نفر فقط برای نمایش یک محیط کاملاً موقت مناسب است و الگوی قابل توصیه برای زیرساخت واقعی نیست. در محیط کاری برای هر نفر حساب مستقل، حداقل سطح دسترسی، ثبت رویدادها و در صورت نیاز `sudo` تعریف کنید. پس از تمرین، کلیدهای آزمایشی را از سرور حذف کنید.

۳.۲ ساخت کلید و برقراری اتصال

فرایند زیر بازسازی آموزشی مراحل کلاس است. مقادیر نمونه‌اند و هیچ IP یا رمز واقعی کلاس در آن درج نشده است.

```
# Generate an SSH key pair on your own computer
ssh-keygen -t ed25519 -C "student-lab"

# View only the public key
cat ~/.ssh/id_ed25519.pub

# Connect after an administrator authorizes the public key
ssh -i ~/.ssh/id_ed25519 USER@SERVER_HOST
```

در مرحله اول، برنامه مسیر ذخیره کلید را می‌پرسد؛ در مرحله بعد می‌توان برای کلید خصوصی Passphrase تعیین کرد. مدرس در Demo از مسیر فایل هویت با گزینه `-i` نیز استفاده کرد. پس از درج کلید عمومی در سرور، اتصال او بدون درخواست رمز حساب سرور انجام شد.

- **تکمیل فنی:** نخواستن رمز حساب به معنای کنار گذاشتن تمام عوامل احراز هویت نیست؛ کلید خصوصی هنوز باید اثبات کند کاربر مجاز است. بهتر است برای کلید خصوصی Passphrase بگذارید و در صورت نیاز از SSH Agent استفاده کنید. محتوای فایلی مانند `id_ed25519` خصوصی است؛ فقط فایل با پسوند `.pub` قابل اشتراک است.

۳.۳ شناخت دستورهای درگیر در Demo

دستور	کارکرد
`ssh`	اتصال به ماشین راه دور
`ssh-keygen`	ساخت زوج کلید
`cat`	نمایش محتوای فایل، از جمله کلید عمومی
`ls -a`	نمایش فایل‌های مخفی مانند `.ssh`
`vi` یا `vim`	ویرایش متن در ترمینال
`touch`	ایجاد فایل خالی یا به‌روزرسانی زمان تغییر
`Ctrl+D`	ارسال پایان ورودی؛ اغلب خروج از Shell

مدرس از تمرین ساخت یک فایل با نام هر دانشجو روی سرور استفاده کرد تا معلوم شود اتصال موفق شده است. این تمرین همچنین نشان می‌دهد SSH یک اتصال تعاملی است و کاربر بعد از ورود می‌تواند همان دستورهای آشنای Linux را اجرا کند.

۳.۴ خودکارسازی چند سرور

پس از تجربه اتصال تعاملی، مدرس اشاره کرد که اعمال تنظیم مشابه روی تعداد زیادی سرور را لازم نیست همیشه دستی انجام دهیم. او از Ansible به‌عنوان نمونه ابزار Configuration Management نام برد، اما وارد آموزش عملی آن نشد. ارتباط این بحث با مهندسی داده آن است که ابزارهای داده اغلب روی چند ماشین نصب و اداره می‌شوند و تکرار دقیق تنظیمات اهمیت دارد.

۴. Git و GitHub: تاریخچه و همکاری

۴.۱ چرا Git به وجود می‌آید؟

مدرس مسئله نسخه‌های متعدد یک فایل را مثال زد: سندی که هر بار با نام‌هایی مانند «نهایی»، «نهایی جدید» و «نهاییِ نهایی» ذخیره می‌شود، خیلی زود پیگیری تغییراتش دشوار می‌شود. Git روشی برای ثبت نسخه‌های معنادار و تاریخچه تغییرات است. هر Commit مشخص می‌کند چه تغییرهایی در یک نقطه از تاریخ پروژه ثبت شده‌اند.

دومین نیاز، همکاری هم‌زمان چند نفر است. در یک پروژه مشترک هر نفر می‌تواند بخشی از کد یا مستندات را تغییر دهد و آن تغییرها در یک روند کنترل‌شده وارد نسخه مشترک شوند. مدرس نمونه مشارکت دانشجویان در مخزن دوره و افزودن نام‌ها از طریق Pull Request را نشان داد.

- **تکمیل فنی:** Git خودِ سیستم کنترل نسخه توزیع‌شده است؛ GitHub و GitLab سرویس‌های میزبانی مخزن و همکاری هستند. امکاناتی مانند Issue و Pull Request از قابلیت‌های پلتفرم‌اند، نه فرمان‌های اصلی Git.

۴.۲ مخزن محلی و مخزن راه دور

مدرس در GitHub یک Repository آزمایشی ساخت و آن را با `git clone` روی سیستم شخصی دریافت کرد. سپس در پوشه ایجادشده فایل‌هایی ساخت و توضیح داد که وجود یک فایل روی کامپیوتر به معنای دیده شدن آن در GitHub نیست.

```
# Clone a repository
git clone REPOSITORY_URL
 cd REPOSITORY_DIRECTORY

# Review the working tree
git status

# Stage and record one intended file
git add example.txt
git commit -m "docs: add example file"

# Send local commits to the remote
git push
```

در نمایش زنده، مدرس دو فایل متفاوت ایجاد کرد و فقط یکی را Stage کرد. همین مثال روشن می‌کرد که `git add` انتخاب تغییرها برای Commit است؛ قرار نیست با هر Commit همه فایل‌های پوشه وارد تاریخچه شوند.

- **تکمیل فنی:** `git add` فایل را به محیط Staging می‌برد، `git commit` تغییرهای Stageشده را در مخزن محلی ثبت می‌کند و `git push` Commitهای محلی را به Remote ارسال می‌کند. برای Commit لازم است هویت نویسنده Git تنظیم شده باشد و برای Push نیز مجوز مخزن وجود داشته باشد.

۴.۳ Git از راه HTTPS یا SSH

مدرس توضیح داد که Git می‌تواند برای ارتباط با سرویس میزبانی از مسیر HTTPS یا SSH استفاده کند. او بخشی از تنظیمات SSH Key در GitHub را نشان داد. مفهوم کلیدی این بود که SSH Key هم برای ورود به سرور Linux و هم برای احراز هویت در GitHub کاربرد دارد، گرچه مقصد و سطح دسترسی هرکدام متفاوت‌اند.

- **تکمیل فنی:** استفاده Git از HTTPS لزوماً به معنای ارسال دستی یک درخواست `curl -X POST` برای هر Push نیست. Git پروتکل تبادل داده مخصوص خود را روی HTTPS یا SSH اجرا می‌کند. برای احراز هویت HTTPS نیز معمولاً از Token یا Credential Manager استفاده می‌شود، نه رمز معمول حساب GitHub.

۴.۴ تغییرات محلی، Stage و Commit

مدرس بارها `git status` و `git diff` را اجرا کرد تا نشان دهد چه چیزی تغییر کرده است. در نمایش کوتاه وضعیت، تغییر یک فایل می‌تواند در ستون مربوط به Working Tree یا ستون مربوط به Index دیده شود. با `git add` تغییر مورد نظر Stage می‌شود. همچنین `git restore --staged` برای خارج کردن اشتباهی یک فایل از Stage نمایش داده شد.

دستور	کاربرد اصلی در مثال مدرس
`git clone`	دریافت کپی اولیه مخزن
`git status`	دیدن وضعیت فایل‌ها و Conflictها
`git status -s`	نمایش خلاصه وضعیت
`git diff`	دیدن تغییرهای هنوز Stageنشده
`git add`	انتخاب تغییرها برای Commit
`git restore --staged`	خارج‌کردن تغییر از Stage بدون پاک‌کردن فایل
`git commit -m`	ثبت یک نسخه در تاریخچه محلی

دستور	کاربرد اصلی در مثال مدرس
`git push`	ارسال Commitها به Remote
`git pull`	دریافت و ادغام تغییرهای Remote با شاخه فعلی

- **تکمیل فنی:** `git diff` به‌صورت پیش‌فرض تغییرهای Working Tree نسبت به Index را نمایش می‌دهد. برای دیدن تغییرهای Stageشده از `git diff --cached` استفاده کنید. همچنین `git restore --staged` به معنی حذف تغییر از فایل اصلی نیست.

۴.۵ Branch، محیط توسعه و Pull Request

مدرس Branch را به‌عنوان راهی برای پیش بردن تغییرهای مستقل معرفی کرد. برای مثال، پیش از قرار گرفتن قابلیت تازه در اختیار کاربران نهایی، تیم آن را جداگانه توسعه و آزمایش می‌کند. سپس با فرایند Review و Merge، تغییرها به شاخه اصلی منتقل می‌شوند.

در GitHub، یک شرکت‌کننده Pull Request فرستاد و مدرس تغییر را بررسی و Merge کرد. این نمایش مفهوم مشارکت در یک مخزن عمومی را از حالت نظری خارج کرد.

- **تکمیل فنی:** Branch الزاماً معادل یک محیط عملیاتی مانند Production یا Test نیست. شاخه یک اشاره‌گر به تاریخچه Commitهاست؛ تبدیل آن به محیط اجرای جداگانه به فرایند استقرار و CI/CD پروژه بستگی دارد.

۵. Git Pull، تعارض‌ها و رفع Conflict

۵.۱ چرا Pull یا Push ممکن است رد شود؟

در آزمایش کلاس، فایلی با نام مشابه هم روی رایانه محلی و هم روی GitHub ساخته شد. نخست `git pull` اخطار داد که تغییر Remote می‌تواند فایل محلی را بازنویسی کند. مدرس نشان داد که قبل از همگام‌سازی باید درباره فایل محلی تصمیم بگیریم، نه اینکه بی‌دلیل همه چیز را پاک کنیم.

در نمونه بعدی، یک فایل مشترک هم در لوکال و هم در Remote ویرایش و Commit شد. هنگام Push, Git به‌دلیل جلوتر بودن تاریخچه Remote، ارسال را رد کرد. سپس با Pull و تلاش برای ترکیب تغییرها، Conflict به وجود آمد.

وضعیت	دلیل احتمالی	رویکرد مناسب
Pull روی فایل Untracked متوقف می‌شود	نسخه Remote قرار است فایل محلی هم‌نام را جایگزین کند	ابتدا فایل را بررسی و در صورت نیاز جابه‌جا یا Commit کنید
Push رد می‌شود	Remote تغییرهایی دارد که در تاریخچه محلی نیست	ابتدا با `fetch` یا `pull` تغییرهای جدید را بررسی کنید
Merge Conflict	Git قادر به انتخاب خودکار محتوای نهایی نیست	نسخه مطلوب را دستی انتخاب یا ترکیب کنید
وضعیت `UU` در `git status -s`	فایل دارای تعارض ادغام حل‌نشده است	متن را اصلاح، Stage و فرایند ادغام را ادامه دهید

- **تکمیل فنی:** حذف فایل محلی تنها راه رفع خطای Pull نیست؛ برای حفظ کار انجام‌شده، نخست `git status` را بررسی و در صورت لزوم از فایل نسخه پشتیبان تهیه کنید. `git reset --hard` و `git clean -fd` می‌توانند داده‌های محلی را از بین ببرند و نباید پاسخ پیش‌فرض به این خطاها باشند.

۵.۲ مشاهده علامت‌های Conflict

مدرس فایل درگیر را باز کرد و نشان داد Git دو نسخه از یک بخش را با نشانه‌های مخصوص مشخص می‌کند. کاربر باید تصمیم بگیرد کدام عبارت بماند یا ترکیبی از هر دو نوشته شود. در این مثال، هدف حفظ اصلاحات مطلوب هر دو نفر بود.

```
<<<<<<< HEAD
Local version
=======
Remote version
>>>>>>> incoming-change
```

این علامت‌ها بخشی از محتوای نهایی پروژه نیستند. بعد از رفع تعارض باید حذف شوند و فایل نهایی از نظر معنا بررسی شود.

۵.۳ روند ایمن رفع Conflict

در این بخش، ترتیب آموزشی از رفتار مشاهده‌شده در کلاس گرفته شده، اما فرمان ادامه ادغام به نوع فرایند بستگی دارد.

```
# Inspect the repository before taking action
git status

# Edit the conflicted file and remove conflict markers
git add example.txt

# If Git is currently performing a merge
git merge --continue

# If Git is currently performing a rebase instead
# git rebase --continue

# Verify the result before uploading
git status
git push
```

- **تکمیل فنی:** `git merge --continue` و `git rebase --continue` جایگزین یکدیگر نیستند. متن خطای Git و خروجی `git status` مشخص می‌کند کدام عملیات در جریان است. اگر Merge معمولی خاتمه یافته یا نیازی به ادامه ندارد، ممکن است تنها یک Commit لازم باشد.

پس از حل تعارض و ارسال Commit، مدرس با Refresh کردن GitHub نشان داد که نتیجه نهایی در Remote نیز دیده می‌شود. این نمایش به دو فایده اصلی Git بازمی‌گشت: ثبت تاریخچه تغییرات و همکاری کنترل‌شده.

۶. جمع‌بندی مجازی‌سازی و Docker

۶.۱ ماشین مجازی چگونه به آزمایش کمک می‌کند؟

در ادامه، مدرس ماشین مجازی آزمایشی قبلی را حذف کرد و نمونه دیگری ساخت تا نصب نرم‌افزار را از ابتدا مرور کند. از طریق پنل ارائه‌دهنده، منابع ماشین مانند CPU و RAM انتخاب می‌شدند. این نمایش یادآور جلسه‌های قبل بود: روی یک میزبان قدرتمند می‌توان چند ماشین مجازی جداگانه ایجاد کرد.

در مقابل، Docker ابزاری برای ساخت و اجرای Container است که با سازوکارهای ایزوله‌سازی سیستم‌عامل کار می‌کند. مدرس از تفاوت VM و Container برای توضیح کاربرد Docker در نصب آسان‌تر سرویس‌ها استفاده کرد.

ویژگی	Virtual Machine	Container
واحد اصلی	ماشین با سیستم‌عامل مهمان	فرایندهای ایزوله‌شده همراه وابستگی‌ها
جداسازی	با Hypervisor و مرز ماشین مجازی	با سازوکارهای Kernel و Runtime
سربار معمول	بیشتر، چون سیستم‌عامل مهمان نیز اجرا می‌شود	معمولاً کمتر
کاربرد کلاس	ساخت سرور آزمایشی	اجرای PostgreSQL روی همان سرور

- **تکمیل فنی:** Container بدون نیاز به سیستم‌عامل مهمان مستقل اجرا می‌شود، اما به Kernel سازگار و Runtime نیاز دارد. روی Windows و macOS, Docker Desktop معمولاً از یک محیط Linux مجازی نیز کمک می‌گیرد؛ بنابراین «Docker هیچ‌وقت VM ندارد» درست نیست.

۶.۲ نصب Docker روی سرور Linux

در Demo مدرس به مستندات نصب Docker مراجعه کرد، فرمان‌های نصب را روی ماشین Ubuntu اجرا کرد و وضعیت را بررسی کرد. در مقطعی دسترسی به مخازن نرم‌افزاری به دلیل محدودیت شبکه دشوار بود و مدرس از راهکار تنظیم DNS نام برد.

- **تکمیل فنی:** تغییر DNS لزوماً محدودیت اتصال HTTPS یا تحریم مخزن را رفع نمی‌کند. قبل از تغییر تنظیمات عمومی سرور، ابتدا DNS Resolution، مسیر شبکه، TLS و دسترسی به Registry را جداگانه آزمایش کنید. استفاده از راهکارهای تغییر مسیر ارتباط نیز باید مطابق مقررات و سیاست سازمان باشد.

۶.۳ Image, Container و اجرای PostgreSQL

مدرس پس از نصب Docker، با دریافت یک Image و اجرای آن، سرویس PostgreSQL را روی ماشین آزمایشی بالا آورد. سپس با `docker ps` کانتینر در حال اجرا را مشاهده کرد. هدف نهایی این بود که از کامپیوتر شخصی با یک Database Client به PostgreSQL متصل شوند و در جلسات بعد روی SQL کار کنند.

مفهوم	معنی در تمرین
Docker Image	بسته مورد نیاز برای ایجاد محیط اجرای PostgreSQL
Container	نمونه در حال اجرای Image
Docker Registry	محل دریافت Image
Environment Variable	راه تنظیم اولیه مانند رمز و نام پایگاه داده
Port Mapping	مشخص کردن مسیر دسترسی میزبان به سرویس داخل کانتینر
`docker ps`	بررسی کانتینرهای در حال اجرا
`docker logs`	دیدن خروجی و خطاهای سرویس

۶.۴ مثال اضافی: PostgreSQL فقط روی Loopback سرور

در Transcript فرمان دقیق کلاس کامل نیست. نمونه زیر یک سناریوی بازسازی‌شده برای آموزش است و برای جلوگیری از قرار دادن پایگاه داده روی اینترنت عمومی، پورت را فقط روی Loopback میزبان منتشر می‌کند.

```
# Create a password interactively, without storing it in shell history
read -rs DB_PASSWORD
printf '\n'

# Run PostgreSQL for a temporary training environment
docker run -d \
  --name training-postgres \
  -e POSTGRES_USER=training_user \
  -e POSTGRES_PASSWORD="$DB_PASSWORD" \
  -e POSTGRES_DB=training_db \
  -p 127.0.0.1:5432:5432 \
  postgres:17

unset DB_PASSWORD

docker ps
docker logs --tail 30 training-postgres
```

این نمونه برای آزمایش کوتاه مناسب است و Volume پایدار برای اطلاعات تعریف نمی‌کند. حذف کانتینر بدون Volume ممکن است داده‌های ذخیره‌شده در آن را نیز از دسترس خارج کند.

- **تکمیل فنی:** تعیین `-p 127.0.0.1:5432:5432` یعنی سرویس از شبکه بیرونی مستقیماً در دسترس قرار نمی‌گیرد. اگر پورت PostgreSQL روی `0.0.0.0` و اینترنت عمومی منتشر شود، حتی با رمز عبور نیز سطح حمله افزایش می‌یابد. برای تمرین از SSH Tunnel یا شبکه خصوصی استفاده کنید.

۷. اتصال به دیتابیس و رفع اشکال شبکه

۷.۱ تفاوت در حال اجرا بودن سرویس و دسترسی از راه دور

در بخش پایانی، مدرس تلاش کرد از رایانه شخصی با یک کلاینت گرافیکی به PostgreSQL سرور وصل شود. مقدارهای لازم شامل IP مقصد، پورت PostgreSQL و نام کاربری و رمز پایگاه داده بودند. او همچنین برای بررسی اتصال شبکه از ابزارهایی مانند `telnet` استفاده کرد.

در ابتدا اتصال برقرار نشد. مدرس خروجی کانتینر، وضعیت شبکه و تنظیم‌های در دسترس را بررسی کرد. این بخش یادآور نکته مهمی بود: اینکه `docker ps` کانتینر را Up نشان دهد، فقط وضعیت اجرای کانتینر را نشان می‌دهد، نه سلامت اتصال از اینترنت به سرویس.

۷.۲ مراحل تشخیص مشکل اتصال

پرسش	بررسی پیشنهادی	نکته
آیا Docker سالم است؟	`docker ps`	کانتینر باید در حال اجرا باشد
آیا خود PostgreSQL آماده است؟	`docker logs training-postgres`	شروع شدن کانتینر ممکن است پیش از آماده بودن DB باشد
آیا پورت روی میزبان منتشر شده؟	`docker port training-postgres`	Port Mapping را با مقدار مورد انتظار تطبیق دهید
آیا پورت روی میزبان پاسخ می‌دهد؟	`ss -lnt`	بررسی سوکت‌های در حال Listen
آیا Firewall یا شبکه ابر محدود کرده؟	قوانین میزبان و پنل ارائه‌دهنده	باز بودن Port Mapping به‌تنهایی کافی نیست
آیا مشخصات DB درست است؟	User, Database و Password	خطای احراز هویت از خطای شبکه جداست

- **تکمیل فنی:** `curl` و `telnet` ابزارهای مناسبی برای بررسی بخش‌هایی از اتصال TCP هستند، اما PostgreSQL با HTTP صحبت نمی‌کند؛ بنابراین نگرفتن پاسخ HTTP از `curl` به‌تنهایی دلیل خرابی دیتابیس نیست. برای آزمون واقعی پروتکل PostgreSQL از `psql` یا یک Database Client استفاده کنید.

۷.۳ مثال اضافی: اتصال امن از رایانه شخصی با SSH Tunnel

در این راه، پورت کانتینر روی Loopback سرور باقی می‌ماند و اتصال از رایانه شخصی با SSH انتقال داده می‌شود. این مثال تکمیل فنی تمرین است، نه فرمان نقل‌شده از مدرس.

```
# Execute on your own computer
ssh -N -L 15432:127.0.0.1:5432 USER@SERVER_HOST
```

سپس در Database Client رایانه شخصی، Host را `127.0.0.1` و Port را `15432` قرار دهید. نام کاربری پایگاه داده در نمونه قبل `training_user` و نام آن `training_db` است. SSH Tunnel باید در مدت اتصال برقرار بماند.

- **تکمیل فنی:** اگر Port Forwarding روی SSH Server غیرفعال باشد، یا PostgreSQL فقط در شبکه دیگری قابل دسترسی باشد، Tunnel به تنظیم متفاوتی نیاز دارد. همین‌طور برای سرویس عملیاتی باید به جای اتصال مستقیم کاربران به پایگاه داده، سیاست دسترسی شبکه و مدیریت Secret سازمان بررسی شود.

۸. گردش کار یکپارچه از سرور تا داده

مدرس می‌خواست دانشجو به‌جای حفظ‌کردن فرمان‌ها، رابطه میان مفاهیم را ببیند. این جریان، نتیجه آموزشی مثال‌های همان جلسه است:

۱. یک VM یا VPS آزمایشی از ارائه‌دهنده زیرساخت دریافت می‌کنیم.

۲. با IP و حساب کاربری، ابتدا از راه SSH به آن وصل می‌شویم.

۳. با SSH Key امکان احراز هویت کلیدی و مدیریت بهتر ورود را می‌فهمیم.

۴. کار با `ls`، `cat`، ویرایشگرها و مسیرهای Linux برای اداره همان ماشین لازم می‌شود.

۵. Git و GitHub تغییرات کد و مستندات تیم را قابل پیگیری می‌کنند.

۶. روی سرور Docker نصب می‌شود و یک Image برای اجرای PostgreSQL استفاده می‌کنیم.

۷. وضعیت اجرای کانتینر و دسترسی به پورت را جداگانه بررسی می‌کنیم.

۸. با کلاینت سازگار و اتصال امن، برای اجرای Query آماده می‌شویم.

این گام‌ها توضیح می‌دهند چرا مجموعه‌ای از موضوع‌های ظاهراً پراکنده، پیش‌نیاز یک پروژه عملی مهندسی داده‌اند.

۹. انتقال تجربه مدرس

مدرس در طول کلاس چند بار بر یادگیری از طریق تمرین مشترک تأکید کرد: دانشجوها کلید عمومی می‌ساختند، روی سرور فایل آزمایشی ایجاد می‌کردند، تغییرها را با Git ثبت می‌کردند و Pull Request می‌فرستادند. او همچنین نشان داد خطاهای اتصال، کندی اینترنت و Conflict بخشی طبیعی از کار عملی‌اند و باید با دیدن وضعیت سیستم و پیام خطا آن‌ها را تشخیص داد، نه با تکرار کورکورانه فرمان‌ها.

تمرین‌های اشاره‌شده در کلاس شامل مشارکت در مخزن دوره، تمرین کار با SSH، و آشنایی با اتصال به پایگاه داده برای جلسه‌های آتی بود. در Transcript مهلت قطعی تازه‌ای برای تحویل تمرین مشخص نشده است.

۱۰. جمع‌بندی

محور جلسه ششم، دیدن ارتباط ابزارها در یک سناریوی واحد بود. Terminal روش اداره سرور است؛ SSH ارتباط امن را برقرار می‌کند؛ Git تاریخچه و همکاری را سامان می‌دهد؛ VM محیط آزمایشی فراهم می‌کند؛ Docker استقرار سرویس را ساده می‌کند و ابزار پایگاه داده برای برقراری ارتباط و اجرای Query به کار می‌رود. در کنار این زنجیره، پیام‌های خطا، سطح دسترسی و امنیت اتصال باید جدی گرفته شوند.

۱۱. سوال‌های مرور

۱. چرا کار با Command Line برای سرورهای Linux مفید است؟

۲. تفاوت SSH با RDP در کاربرد معمول این جلسه چیست؟

۳. Public Key و Private Key در SSH چه تفاوتی دارند؟

۴. فایل `authorized_keys` چه نقشی دارد؟

۵. سه گام `git add`، `git commit` و `git push` چه تفاوتی دارند؟

۶. Git چه تفاوتی با GitHub دارد؟

۷. کاربرد `git status` و `git diff` چیست؟

۸. چرا `git push` ممکن است رد شود؟

۹. هنگام Merge Conflict چه تصمیمی باید گرفته شود؟

۱۰. Branch و Pull Request چه نقشی در همکاری دارند؟

۱۱. تفاوت VM و Container چیست؟

۱۲. `docker ps` چه چیزی را اثبات می‌کند و چه چیزی را نه؟

۱۳. برای اتصال به PostgreSQL چه اطلاعاتی نیاز داریم؟

۱۴. چرا باز بودن پورت روی کانتینر لزوماً به معنای دسترسی از اینترنت نیست؟

۱۵. در سناریوی کلاس، SSH و Docker و Git چه ارتباطی با مهندسی داده پیدا کردند؟

۱۲. پاسخ‌نامه مختصر

۱. اجرای فرمان و خودکارسازی کارهای مدیریت سرور بدون سربار محیط گرافیکی را ساده‌تر می‌کند.

۲. SSH عمدتاً برای اتصال امن به خط فرمان و RDP برای دسترسی گرافیکی از راه دور است.

۳. کلید عمومی به مقصد داده می‌شود؛ کلید خصوصی نزد مالک می‌ماند.

۴. فهرست کلیدهای عمومی مجاز برای ورود به یک حساب روی سرور را نگه می‌دارد.

۵. `add` انتخاب برای Stage، `commit` ثبت محلی و `push` ارسال به Remote است.

۶. Git سیستم کنترل نسخه است و GitHub پلتفرم میزبانی و همکاری روی مخازن Git.

۷. `status` وضعیت فایل‌ها و `diff` تفاوت‌های محتوایی را نشان می‌دهند.

۸. چون Remote دارای Commitهایی است که هنوز در تاریخچه محلی وجود ندارند، یا مجوز ارسال کافی نیست.

۹. محتوای نهایی باید با بررسی هر دو تغییر مشخص و Conflict Markerها حذف شوند.

۱۰. Branch مسیر تغییر مستقل است و Pull Request پیشنهاد بررسی و ادغام تغییرها را فراهم می‌کند.

۱۱. VM سیستم‌عامل مهمان مستقل دارد؛ Container معمولاً Kernel میزبان را به اشتراک می‌گذارد.

۱۲. اجرای کانتینر را نشان می‌دهد، نه آمادگی سرویس یا باز بودن مسیر شبکه از راه دور.

۱۳. Host, Port، نام پایگاه داده، User, Password یا روش احراز هویت مناسب.

۱۴. Firewall، قوانین شبکه ارائه‌دهنده، Port Mapping و آدرس Listen می‌توانند مانع شوند.

۱۵. در پروژه داده باید سرور را اداره، کد را هماهنگ، سرویس را اجرا و از اتصال سالم پایگاه داده مطمئن شد.

۱۳. واژه‌نامه تخصصی

اصطلاح انگلیسی	معادل یا تعریف کاربردی فارسی
CLI	رابط خط فرمان
Server	سرور یا رایانه خدمت‌دهنده
Cluster	مجموعه ماشین‌های هماهنگ
VPS / VM	سرور خصوصی مجازی / ماشین مجازی
SSH	پروتکل دسترسی امن از راه دور

Host Key Fingerprint	اثرانگشت کلید هویت سرور
Public Key	کلید عمومی
Private Key	کلید خصوصی
Passphrase	عبارت محافظ کلید خصوصی
Authorized Keys	فهرست کلیدهای عمومی مجاز
Configuration Management	مدیریت و اعمال هماهنگ تنظیمات
Git	سیستم کنترل نسخه توزیع‌شده
Repository	مخزن فایل و تاریخچه
Clone	دریافت نسخه محلی مخزن
Working Tree	فایل‌های کاری موجود در پوشه پروژه
Staging Area / Index	محل انتخاب تغییرها برای Commit
Commit	ثبت تغییر در تاریخچه محلی
Remote	مخزن متصل در سمت دیگر
Push / Pull	ارسال / دریافت و یکپارچه‌سازی تغییرها
Branch	شاخه مستقل تاریخچه توسعه
Pull Request	درخواست بررسی و ادغام تغییر
Merge Conflict	تعارض در ترکیب تغییرها
Docker Image	تصویر آماده اجرای سرویس
Docker Container	نمونه اجرایی Image
Port Mapping	نگاشت پورت میزبان و کانتینر
PostgreSQL	سامانه مدیریت پایگاه داده رابطه‌ای
Database Client	ابزار اتصال و کار با پایگاه داده
SSH Tunnel	انتقال اتصال شبکه از طریق SSH